

TRABAJO FINAL — USO DE IA GENERATIVA

Proyecto Myco

Aplicación web de recomendación de hongos adaptógenos, construida mediante vibe coding con IA generativa.

Diplomatura en IA Aplicada a Entornos Digitales de Gestión

FCE-UBA · Cohorte 2026

Julieta Speranza

julietasperanza@hotmail.com · 341-3443229

21SP30100785@campus.economicas.uba.ar

a. Introducción

Elegí este tema porque...

...me apasiona el mundo de los hongos y todo su potencial para mejorar la salud y el bienestar de las personas. Me interesa especialmente la aplicación de los hongos adaptógenos como una alternativa natural para acompañar el equilibrio físico y mental.

Creo que hoy existe una tendencia a buscar soluciones cada vez más artificiales para problemas cotidianos, cuando muchas respuestas pueden encontrarse en la naturaleza. Me inspira la idea de volver a las raíces, recuperar conocimientos ancestrales y acercar esos saberes de una forma accesible, simple y basada en evidencia.

Con este proyecto me gustaría contribuir a difundir los beneficios de los hongos adaptógenos mediante una experiencia digital que no solo recomiende un producto, sino que también invite a las personas a conectar con su bienestar de una manera lúdica, educativa y consciente.

Objetivos

- Diseñar una aplicación web que, a partir de un breve cuestionario, recomiende un hongo adaptógeno (Reishi, Cordyceps, Melena de León o Turkey Tail) acorde a lo que la persona necesita.
- Mostrar en un mapa los puntos de venta cercanos donde conseguir el hongo recomendado.
- Cerrar la recomendación con una experiencia interactiva temática (una playlist, un juego de reflejos, un memotest o un juego de defensa), para que el recorrido no termine en un dato frío sino en una vivencia.
- Construir la solución sobre infraestructura gratuita y accesible (Google Apps Script + Google Sheets), sin necesidad de hosting pago ni de conocimientos previos de programación.
- Garantizar que la recomendación sea transparente y auditable (reglas fijas, no un modelo de IA en producción) y que la aplicación deje explícito, en todo momento, que no reemplaza una consulta médica.

b. Marco conceptual

El proyecto se desarrolló íntegramente mediante vibe coding: una conversación en lenguaje natural con un modelo de IA generativa, a partir de la cual se generó, probó y corrigió el código de la aplicación, sin escribir código a mano.

Herramienta principal: Claude (Anthropic)

Se utilizó Claude, en su interfaz de chat, para las siguientes tareas:

- Redacción del documento de especificación técnica del proyecto (arquitectura, modelo de datos, flujos de usuario) a partir de una idea descripta en lenguaje coloquial.

- Generación completa del código: backend en Google Apps Script (autenticación, base de datos en Sheets, motor de recomendación, envío de correos) y frontend en HTML/CSS/JavaScript (pantallas, estilos, cuatro mini-juegos).
- Diseño de la identidad visual (paleta de colores, tipografías, animaciones) siguiendo un concepto solicitado explícitamente: "naturaleza, bienestar, vida saludable y volver a la tierra".
- Generación de mockups visuales interactivos de cada pantalla antes de programarla, para validar el diseño con la persona usuaria sin gastar tiempo de desarrollo en algo que después había que rehacer.
- Generación del ícono de la aplicación de forma programática (dibujo vectorial de un hongo con la paleta de marca).
- Diagnóstico y corrección de errores a partir de capturas de pantalla reales tomadas por la persona usuaria en su celular.

Plataforma de ejecución: Google Apps Script + Google Sheets

No es una herramienta de IA generativa, pero es la plataforma elegida (por decisión propia, no de la IA) para alojar la aplicación: permite publicarla como una Web App gratuita, usando una planilla de Google Sheets como base de datos, sin contratar hosting ni servidores.

Uso deliberado de IA solo en el desarrollo, no en producción

Una decisión de diseño explícita, tomada desde el primer prompt, fue que la aplicación terminada no llama a ningún modelo de IA en tiempo de ejecución: la recomendación se calcula con un sistema de puntajes por reglas fijas, totalmente auditable. La IA generativa se usó únicamente como herramienta de construcción, para que la persona que use la app finalmente no dependa de ningún costo de tokens ni de conexión a un modelo externo.

c. Metodología

El proceso fue iterativo: cada entrega se instaló y se probó en un celular real, y las capturas de pantalla de esas pruebas fueron el insumo principal para la siguiente ronda de ajustes.

1. Prompt inicial

El proyecto arrancó con un prompt que describía la idea completa en lenguaje natural, incluyendo la justificación personal del tema, el funcionamiento deseado (cuestionario → recomendación → mapa → experiencia gamificada), la restricción técnica (Apps Script + Sheets), el requisito de registro con contraseña temporal, y la decisión de no usar IA en producción. Un fragmento textual del prompt:

"Una aplicación web que te recomiende qué hongos adaptógenos consumir para mejorar el rendimiento, salud y bienestar. A partir de responder algunas preguntas te recomienda qué hongo necesitas, y luego te muestra en el mapa qué lugares los venden [...] No va a requerir IA ni modelos para la recomendación. Vamos a usar IA para generarla, armar la aplicación y luego, en producción, quien la use no necesita gastar tokens para usarla."

A partir de este prompt, Claude generó el documento de especificación completo (arquitectura, modelo de datos, plan de fases) antes de escribir una sola línea de código, lo cual permitió validar el alcance antes de invertir tiempo de desarrollo.

2. Identidad visual antes del código

Se pidió explícitamente un diseño "pensado en la naturaleza, el bienestar, la vida saludable y volver a la tierra". Claude propuso una paleta (verde bosque, arena, oro de espora), tipografías (Fraunces, Work Sans, JetBrains Mono) y un elemento de firma visual (una línea de raíz de micelio animada), que se aprobaron antes de generar el código de las pantallas.

3. Generación del código y mockups previos a cada cambio grande

Para cambios de alto impacto visual (por ejemplo, rediseñar el mini-juego de Turkey Tail), en vez de programar directamente se le pidió a Claude generar previews visuales interactivas de varias alternativas. Se compararon tres propuestas de juego (un escudo que bloquea partículas, un juego de memoria de anillos concéntricos, y un balancín de equilibrio) y recién después de elegir una se programó, evitando descartar trabajo ya hecho.

4. Qué decidió la persona y qué resolvió la IA

- Decisiones propias: el tema y su justificación, los cuatro hongos y sus experiencias asociadas, la restricción de plataforma, la regla de no usar IA en producción, la dirección estética, cuál de las tres propuestas de juego implementar, la redacción final de los textos, y la aprobación o rechazo de cada entrega en base a pruebas reales.
- Resuelto por la IA: la arquitectura completa, el esquema de la base de datos, la lógica de autenticación (hash de contraseñas, sesiones, recuperación), el motor de recomendación por puntajes, el código de los cuatro mini-juegos, el sistema de diseño (CSS), el ícono de la aplicación, y el diagnóstico y la corrección de errores a partir de capturas de pantalla.

5. Qué salió mal y cómo se corrigió

Durante las pruebas en dispositivo real aparecieron varios problemas concretos, todos identificados a partir de capturas de pantalla y corregidos en la ronda siguiente:

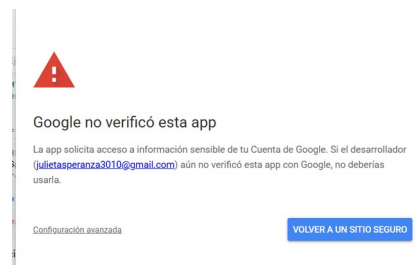


Figura 1. Aviso de Google al abrir la app por primera vez ("Google no verificó esta app"), esperable en cualquier Web App personal de Apps Script; se resolvió explicando el flujo de autorización avanzada.

- Código visible en pantalla: los archivos de los mini-juegos mostraban literalmente el texto "<?!= include(...) ?>" en vez de renderizar el contenido. La causa fue un error real en la función include() de Apps Script, que no evaluaba las plantillas anidadas; se corrigió cambiando el método de renderizado del lado del servidor.
- El juego de defensa ("Escudo del bosque") terminaba de forma abrupta y parecía "reiniciarse solo": la pantalla de "juego terminado" se dibujaba y, en el mismo ciclo de animación, se volvía a tapar con el frame siguiente del juego. Se corrigió agregando una condición que detiene el redibujado una vez que el juego finalizó.

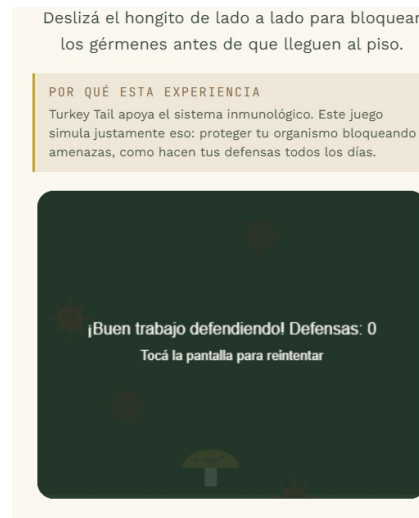


Figura 2. Evidencia del juego terminando antes de lo esperado, reportada por la persona usuaria y usada para ajustar la dificultad y la duración.

- Carrera de navegación: si se tocaba "Crear cuenta" o "Ya tengo cuenta" en el primer instante de carga, la verificación de sesión guardada (que corre en paralelo, de fondo) podía resolverse después y redirigir a la persona al panel principal, pisando la navegación manual. Se corrigió con una bandera que cancela la redirección automática si ya hubo una navegación manual del usuario.
- El selector de avatar y el mapa geolocalizado no reflejaban los cambios esperados en una primera entrega, porque un archivo con la lógica correspondiente no había sido reemplazado por completo en el editor de Apps Script; se resolvió reforzando, en cada entrega, una lista explícita de qué archivo crear y cuál reemplazar.

d. Resultados

El resultado es una aplicación web funcional, publicada como Web App de Google Apps Script, con las siguientes características:

- Registro propio de usuarios con contraseña temporal enviada por correo y cambio obligatorio en el primer ingreso, más recuperación de contraseña olvidada.
- Sesión persistente: no vuelve a pedir inicio de sesión en cada visita.

- Cuestionario de 5 preguntas con motor de recomendación por puntajes (sin IA en producción).
- Resultado enriquecido: descripción del hongo, 5 beneficios clave, hábitos diarios sugeridos y una recomendación de dosis y consumo, con la aclaración de que no reemplaza indicación profesional.
- Mapa que prioriza los puntos de venta cercanos a la ciudad cargada en el perfil del usuario.
- Cuatro experiencias gamificadas: una playlist para Reishi, un juego de reflejos ("runner") para Cordyceps, un memotest para Melena de León, y un juego de defensa ("Escudo del bosque") para Turkey Tail.
- Perfil editable (nombre, edad, ciudad y un avatar de hongito a elección entre 5 opciones), visible en la barra superior de toda la aplicación.
- Un aviso cálido, disponible en cualquier momento desde el panel principal, aclarando que la aplicación no reemplaza una consulta médica.
- Ícono propio de la aplicación para cuando se agrega a la pantalla de inicio del celular.

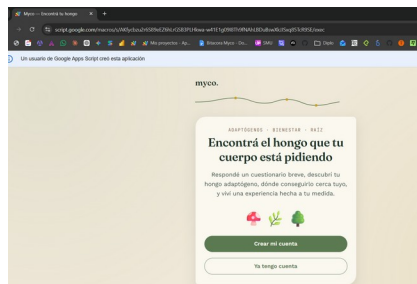


Figura 3. Primera versión funcionando: pantalla de bienvenida cargando correctamente sobre Apps Script.



Figura 4. Pantalla de bienvenida en su versión final, con el texto revisado en una iteración posterior.



Figura 5. Pantalla de resultado enriquecida, con beneficios clave y buenas prácticas sugeridas.



Figura 6. Pantalla de perfil, con selector de avatar, edad y ciudad.

e. Análisis crítico

Fortalezas

- Permitió a una persona sin formación técnica describir un problema en lenguaje cotidiano y obtener, a cambio, una aplicación funcional de punta a punta.
- El ciclo de iteración fue muy rápido: cada corrección o pedido de cambio se resolvía en minutos, y las capturas de pantalla del celular real funcionaron como un canal de feedback efectivo.
- Costo de infraestructura nulo: toda la aplicación corre sobre una cuenta de Google gratuita.
- La lógica de recomendación es transparente y auditable (reglas fijas visibles en el código), en vez de depender de un modelo de caja negra.
- El tono cálido y los límites éticos (aviso médico reabrible en cualquier momento) se sostuvieron a lo largo de toda la iteración, no solo en la primera pantalla.

Limitaciones

- Techo de la plataforma: el aviso "Un usuario de Google Apps Script creó esta aplicación" no se puede quitar sin pasar por un proceso de verificación de OAuth de Google, inviable para un proyecto personal sobre una cuenta de Gmail.

- Google Sheets no es una base de datos pensada para escalar a muchos usuarios concurrentes ni ofrece controles de acceso granulares.
- Algunos errores (por ejemplo, el de los includes anidados) solo se manifestaron al probar en el entorno real de Apps Script, no eran predecibles de antemano; esto obligó a varias rondas de prueba-corrección que hubieran sido evitables con un entorno de testing más cercano a producción.
- El envío de correos vía MailApp tiene un límite diario de envíos según el tipo de cuenta de Google, algo a tener en cuenta si la aplicación creciera en usuarios.

Oportunidades

- Migrar la persistencia a una base de datos dedicada si el proyecto necesitara escalar más allá de un uso personal o de demostración.
- Sumar la API de Google Maps con clave propia para mostrar múltiples puntos de venta reales en un mismo mapa interactivo.
- Ampliar el catálogo de hongos adaptógenos y sumar un historial de recomendaciones a lo largo del tiempo.

Evaluación AIBPS

Ágil: el proceso de construcción fue muy ágil — cada ronda de feedback (una captura de pantalla y una frase describiendo el problema) se tradujo en una corrección concreta en minutos, y los mockups previos evitaron reprogramar funcionalidades ya construidas.

Fluida: la experiencia final del usuario es en general fluida (sesión persistente, formularios pre-cargados, navegación clara), aunque con un punto de fricción real: el aviso de Google al abrir la app por primera vez, propio de la plataforma elegida y no eliminable desde el código.

Protegida: las contraseñas se guardan con hash y sal por usuario, nunca en texto plano; los datos escritos en la planilla se sanitizan para evitar inyección de fórmulas; y la contraseña temporal obliga a un cambio en el primer ingreso. La limitación honesta es que Google Sheets, como base de datos, no ofrece el mismo nivel de control de acceso que un motor de base de datos dedicado.

Bajo control humano: la recomendación de hongos es 100% basada en reglas fijas y auditables, sin ningún modelo de IA corriendo en producción; los datos de cada usuario quedan en una planilla que la persona dueña del proyecto puede inspeccionar o corregir en cualquier momento; y la aplicación incluye, de forma reabrible y no solo al inicio, una aclaración explícita de que no reemplaza a un profesional de la salud.

f. Conclusiones

Aprendizajes

- El vibe coding permite a una persona sin experiencia en programación llevar una idea hasta una aplicación funcional, siempre que se sostenga un ciclo corto de prueba en dispositivo real y feedback concreto (capturas de pantalla, no solo descripciones).
- Pedir mockups visuales antes de programar cambios grandes ahorró tiempo de desarrollo y evitó descartar trabajo ya hecho, en especial al decidir entre alternativas de diseño.
- Los errores más difíciles de anticipar fueron los específicos de la plataforma (Apps Script), no los de la lógica de la aplicación en sí; probar en el entorno real fue indispensable para encontrarlos.

Recomendaciones futuras

- Mantener siempre visible y accesible el aviso de que la aplicación no reemplaza una consulta médica, tal como se implementó, ante cualquier extensión futura del proyecto.
- Si el proyecto crece más allá de una demostración personal, evaluar migrar la base de datos y el hosting fuera de Apps Script para ganar escalabilidad y controles de seguridad más robustos.
- Documentar cada iteración (como se hizo en este informe) para poder mostrar el proceso completo, no solo el resultado final, ya que gran parte del valor del trabajo está en cómo se dirigió a la IA y se corrigieron los problemas encontrados.